`fca_math.py`: a Python package for Formal Concept Analysis

Bernhard Ganter

Version -1 as of June 30, 2026

This document is an introduction to my Python package for Formal Concept Analysis. Read the introduction to understand what it may be useful for, and what not.

You prefer to try it out first? You should have Python3 installed and the `fca_math` package in your working directory. After starting your Python shell, loading it can be done by typing

```
>>> from fca_math import *
```

Then type Todo

```
>>> intro()
```

Or scroll through Figures 1, 2, 3, etc., which list frequently used commands.

Contents

1 Introduction **3**

2 Licence, style and project setup **3**

3 Quick intro: start with a formal context **3**
- 3.1 Working with sets . . . 3
- 3.2 The `Context` class . . . 4
- 3.3 Names and numbers . . . 6
- 3.4 Scales: Predefined formal contexts . . . 8
- 3.5 Modifying a formal context . . . 11
- 3.6 Direct access to attributes of the `Context` class . . . 12
- 3.7 Clarifying, reducing, and enlarging a formal context . . . 13
- 3.8 Create a new context from a given one . . . 14
- 3.9 Create a new context from two given ones . . . 15
- 3.10 Extents and intents . . . 17

4 Examples **18**
- 4.1 Counting monotone Boolean functions . . . 18

4.2 Do integer partition lattices have congruences? 19
4.3 Size of an order relation 19

5 More on formal contexts **20**
5.1 Properties of formal contexts and of concept lattices 20
5.2 Generators and such 20
5.3 Bonds and closed relations 21

6 Closure operators **21**
6.1 The `Closureoperator` class 21
6.2 Making a formal context from a closure operator 23
6.3 Generators 23
6.4 Example 23

7 Diagrams with given placement **24**
7.1 For ConExp users 26

8 Additive diagrams **26**

9 Implications **29**
9.1 The `Implist` class 30
9.2 Closure by implications 30
9.3 Example 31

10 Example: What do we need to prove? **32**

11 Exploration **34**

12 Many-valued contexts and conceptual scaling **34**

13 Computing placements for diagrams **34**

Index **34**

1 Introduction

This software package contains parts of my software collection. I am Bernhard Ganter, co-author of the standard monograph[1] and other books on Formal Concept Analysis. I have accompanied the development of this research area from the very beginning. Recently, I started to port parts of my software, which was developed over many years, from `Pascal` to `Python`. This is a first version.

My implementation is not particularly fast. I have already been told that it is not suitable for professional data analysis. But that is not what it is for. There are already a considerable number of Python programs for Formal Concept Analysis, and some of them may be more efficient. My interest lies in the mathematical foundations, and this collection of programs allows me to explore many of the theoretical results without much effort.

2 Licence, style and project setup

At the moment, this collection is my property and therefore not free software in the strict sense. You are free to use and distribute it, but please do not make any substantial changes without consulting me. However, this is only intended to be temporary, until a certain quality standard is reached. Unless otherwise specified, the GPLv2 licence will apply from 1 January 2027.

Why? Formal Concept Analysis, the mathematical theory for which this software is written, is remarkably successful. One of several reasons is that it uses a fairly consistent language and adheres to some style rules, as they are realised in the aforementioned textbook[1]. This software is intended to accompany that book. Contributions are very welcome, but are asked to adhere to the standard language and rules. There is also a free text entitled "The `fca_math` enlightener" that introduces both the mathematics of Formal Concept Analysis and the Python package.

Another reason why this software is not free software for the time being is the fact that it is a weakly structured and still very incomplete collection which lacks the qualities of a solid Python project with fully formulated setup tools. This is also the reason for scheduling the release for 2027. I hope that a better-maintained version will be available by then. Again, contributions and advice are welcome!

3 Quick intro: start with a formal context

3.1 Working with sets

Formal Concept Analysis uses the language of set theory. That is why our programs also work with sets. You certainly know how sets are noted down in mathematics: the elements are written between curly brackets, separated by

[1] B. Ganter, R. Wille: *Formal Concept Analysis – Mathematical Foundations*, 2nd ed. (2024), Springer, doi:10.1007/978-3-031-63422-2

commas. For example, $\{1, 2, 3, 6\}$ denotes the set of all divisors of the natural number 6.

However, there is a small problem here. With a set, the order of its elements is irrelevant. But what you have written down has an order. $\{2, 6, 1, 3\}$ is obviously a different *text*. But that doesn't matter as long as you know that a set is meant and that the order in which it is written down is arbitrary.

This is also the case in our programs. According to the mathematical definition, *sets* are actually processed, but internally these are usually represented as *lists*. This proves to be unproblematic, and often advantageous and convenient. More information on this is provided in Section 3.3.

3.2 The `Context` class

To be able to do something Formal Concept Analysis, you need a formal context. Choose some variable name,[2] for example `kk`. If you enter

```
>>> kk = Context(3,4)
```

then Python creates an "empty" formal context with 3 objects and 4 attributes under the name `kk`. You can have this context displayed. The command

```
>>> print(kk)
```

serves this purpose. It produces the following output:

```
\begin{cxt} % 3 objects, 4 attributes
  \cxtName{}
  \att{0} % 0
  \att{1} % 1
  \att{2} % 2
  \att{3} % 3
  \obj{....}{0} % 0
  \obj{....}{1} % 1
  \obj{....}{2} % 2
\end{cxt}
```

The information is easy to read, but the format is perhaps somewhat unusual. This is LaTeX-code[3] (for the `fca.sty` LaTeX package) defining

	0	1	2	3
0				
1				
2				

The table indeed symbolizes a formal context with three objects and four attributes, but an empty one, without incidences. With

[2] We often use `kk`, but any valid `Python` variable name can be used for formal contexts.

[3] The document you are reading is written using the LaTeX typesetting system commonly used by scientists. There is an additional package for this, `fca.sty`, which simplifies the writing of texts on Formal Concept Analysis.

```
>>> kk.make_incident(0,1)
```

one gets

	0	1	2	3
0		×		
1				
2				

There are easier ways: for example, you can enter the incidence relation line by line:

```
>>> kk.set_intent(0,'.x.x')
>>> kk.set_intent(1,'.xx.')
>>> kk.set_intent(2,'x.x.')
```

This defines `kk` as

	0	1	2	3
0		×		×
1		×	×	
2	×		×	

With

```
>>> kk.print_incidence()
```

you get a simpler output, showing only the incidence relation:

```
.x.x
.xx.
x.x.
```

You may also invoke

`kk=cxt_typein()`. Then you will be asked for all details of the formal context, step by step. Object intents must be given in the form `xx.x..x.`, without quotion marks.

One can already do something with this little formal context, e.g. by typing

```
>>> kk.number_of_concepts()
```

The result is 7. Want to see these concepts? Type

```
>>> kk.twenty_concepts()
```

and get

```
0 : ( {0, 1, 2} , set() )
1 : ( {1, 2} , {2} )
2 : ( {0, 1} , {1} )
3 : ( {0} , {1, 3} )
4 : ( {1} , {1, 2} )
5 : ( {2} , {0, 2} )
6 : ( set() , {0, 1, 2, 3} ) .
```

To save this formal context, enter

```
>>> kk.write('somefilename')
```

You will find a file `somefilename.cxt` in your context directory `fca_math/contexts/user`. To load this file and to give it an arbitrary variable name (say, `kk2`), type

```
>>> kk2 = Context.read('somefilename')
```

To summarize:

`kk = Context(g,m)` creates a formal context with `g` objects, `m` attributes, having an empty incidence relation, and assigns it to the variable name `kk`.

`kk = Context.read('filename')` reads a context in Burmeister format from `filename.cxt`. The second line of that file may contain a context name. To manually give in all details of a new formal context, invoke

`kk = cxt_typein()`.

`print(cc)`, where `cc` is a variable of class `Context`, prints the context in LaTeX `fca.sty` format. If you prefer to see only the incidence relation, type

`cc.print_incidence()`.

Every now and then you should save your context `kk` to `filename.cxt`:

`kk.write('filename')`.

3.3 Names and numbers

A formal context (G, M, I) has two *carrier* sets: the set G of its objects and the set M of attributes. The order of their elements is irrelevant, but internally we store both G and M as *lists*, i.e., impose a linear order on them. When a formal context is represented as a `Python` object `kk`, then each of its objects and each of its attributes has *two* names: its *name*, as represented in the list `kk.gname` (or `kk.mname`, respectively), and its list *index*.

As long as you work in interpreter mode, i.e., in the interactive `Python` shell, both representations may be used interchangably. In script mode, i.e. when writing programs, only the indices are used. Note that indices are *numbers* while names must be *strings*.

However, this can also be quite confusing and takes some getting used to. So consider the following formal context, with variable name `cc`, as an example.
It seems that there is a misplaced cross: the number 3 is, of course, not a square, but a prime number.
If we try to correct this by typing

	odd	prime	square
1	×		×
2		×	
3	×		×
4			×

```
>>> cc.make_incident(3,prime)
>>> cc.make_nonincident(3,square)
```

we get error messages such as

```
>>> NameError:  name 'prime' is not defined.  Did you mean:  'print'?
```

`Python` expects as input here either a string or a number, and we have use quotation marks, i.e., to type `'prime'` and `'square'` instead. But that is not enough. Entering

```
>>> cc.make_incident(3,'prime')
```

will give the number 4 the attribute *prime*, not the number 3. The reason is that the list of object names is `['1', '2', '3', '4']`, and the element with the index 3 in this list is '4'. What we need to enter in order to correct the false cross is either

```
>>> cc.make_incident('3','prime')
>>> cc.make_nonincident('3','square')
```

or, equivalently,

```
>>> cc.make_incident(2,'prime')
>>> cc.make_nonincident(2,'square') ,
```

but also equivalent is

```
>>> cc.make_incident(2,1)
>>> cc.make_nonincident(2,2) .
```

For `make_incident`, `make_nonincident`, and `isincident`, both input types are allowed, and also for the two `flip` methods described below. And this is even true for the `Context` class itself. To define the above formal context `cc` (in its corrected version), one can also write

```
>>> cc = Context(['1','2','3','4'],['odd','prime','square'])
>>> cc.make_incident('1','odd')
>>> cc.make_incident('1','square')
>>> cc.make_incident('2','prime')
>>> cc.make_incident('3','odd')
>>> cc.make_incident('3','prime')
>>> cc.make_incident('4','square')
```

To change the numbering, use

`kk2 = kk1.gflip(g,h)`, where `g` and `h` are object numbers. The "new" context `kk2` is, mathematically spoken, the same as `kk1`, but the numbering is different. The analogous method exists for attributes:

`kk2 = kk1.mflip(m,n)`.

We extend the idea of using both names and numbers for the elements, and combining them in a list called a 'carrier', not only for formal contexts but also for other data types such as closure operators and implication lists. The following three methods mediate between names and numbers:

`carrier,size = set_carrier(number_or_list)` expects as input parameter `number_or_list` either a list of names, like for example `['Curia', 'Temple of Vesta', 'Arch of Titus']` or a number n, which then is replaced by the list `['0','1',...,'n-1']`. The return value consists of two parts. The first, `carrier`, contains the list, the second, `size`, gives the length of the list.

`names = named_set(carrier,inset)` interpretes the `inset` as a set of indices of `carrier` and returns the corresponding set of names.

`indices = denamed_set(carrier,inset)` interpretes the `inset` as a set of names in the `carrier`-list and returns the corresponding set of indices.

`indices = denamed_unknown(carrier,inset)` interpretes `inset` as a set of indices, names, or as an `x.xx..` sequence and outputs the set of corresponding indices.

3.4 Scales: Predefined formal contexts

It is often unnecessary to type a context in full, as there are many predefined contexts, called `Scales`, and there are modification methods. For example,

```
>>> kk = Scale.interordinal(3)
```

generates the interordinal scale on three objects and assigns it to the variable named `kk`. After reducing it by

```
>>> kk = kk.reduce()
```

one obtains

$\mathbb{I}_3$-r	≤ 1	≤ 2	≥ 2	≥ 3
1	×	×		
2		×	×	
3			×	×

This is the same incidence relation as that of `kk` above, except that the first and the last column are interchanged. But now there are meaningful names for objects, attributes, and for the context itself.

A list of the recently implemented context families is shown in Figure 1. Below, we explain what these names mean. Our nomenclature follows that of the aready cited book by Ganter & Wille. There you also find precise definitions.

`kk = Scale.nominal(n)` creates a nominal scale for n.

`kk = Scale.contranominal(n)` creates a contranominal scale for n.

`kk = Scale.ordinal(n)` creates a ordinal scale for n.

`kk = Scale.interordinal(n)` creates a interordinal scale for n.

`kk = Scale.biordinal(m,n)` creates a biordinal scale for m, n.

- `Context(g,m)`
- `Scale.nominal(n)`
- `Scale.contranominal(n)`
- `Scale.ordinal(n)`
- `Scale.interordinal(n)`
- `Scale.integer_partitions(n)`
- `Scale.set_partitions(n)`
- `Scale.noncrossing(n)`
- `Scale.tamari(n)`
- `Scale.permutations(n)`
- `Scale.additively_saturated(n)`
- `Scale.pg2(d)`
- `Scale.dichotomic()`
- `Scale.biordinal(m,n)`
- `Scale.contraordinal(rel)`
- `Scale.max_antichains(rel)`
- `Scale.general_interordinal(rel)`
- `Scale.convex_ordinal(rel)`

Figure 1: List of predefined formal contexts

`kk = Scale.dichotomic()` creates the dichotomic scale.

The partitions of any positive integer n from a lattice. A formal context for this is given by

`kk = Scale.integer_partitions(n)`.

`kk = Scale.tamari(n)` returns the formal context for the Tamari lattice of bracketings a a word of length n.

`kk = Scale.set_partitions(n)` yields the formal context for the partition lattice of an n-element set.

`kk = Scale.noncrossing(n)` yields the formal context for the lattice of non-crossing partitions of a regular n-gon.

`kk = Scale.permutations(n)` yields the formal context for the lattice of all permutations of an n-set.

A subset $S \subseteq \{0, 1, \ldots, n-1\}$ is *additively saturated*, if from $a, b \in S$ and $a + b < n$ it always follows that $a + b \in S$. The saturated subsets form a closure system. The lattice of closed sets is obtained from

`Scale.additively_saturated(n)`

`kk = Scale.pg2(d)` yields the formal context for the lattice of all subvectorspaces of $GF(2)^d$.

Slightly different is

`kk = Scale.contraordinal(rel)`, because it expects as input parameter `rel` a "relation", which here is nothing but a formal context having the same number of objects and attributes. The method generates the contra(quasi)ordinal scale for the reflexive transitive closure of `rel`. Closely related is

`kk = Scale.max_antichains(rel)`, but here `rel` should not contain non-trivial cycles. Then the maximal antichains of the ordered set generated by `rel` are precisely the sets of the form $A \cap A'$, where A is an extent of `kk`.

`kk = Scale.general_interordinal(rel)`

`kk = Scale.convex_ordinal(rel)` make `kk` the *convex-ordinal scale* for `rel`, the extents of which are precisely the convex subsets of the ordered set given as `rel`.

3.4.1 Auxiliary

In the subsections named "Auxiliary'', we present Python functions that we have implemented as auxiliary functions, but which are probably not of general interest.

`kk = Scale.reverse_ordinal(n)` creates a reverse ordinal scale for n.

`setvariable = setinput(whatever)`, a function which allows to input sets not only in the usual form as $\{a, b, \ldots\}$, but also as strings of `'x'`es and `'.'`s, such as 'x.x..xx'.

`wrd = kk.xdot_out(g)` outputs the object intent `g'` as a string of `'x'`es and `'.'`s to a variable, which in this example has the name `wrd`.

`setvar = inversions(string)` returns the set of *inversions* of `string`, i.e., of pairs (`s[i]`,`s[j]`), where `ord(s[i]) > ord(s[j])`, but `i < j`. Needed for the context of `permutations`.

3.5 Modifying a formal context

By 'modify' we mean here changes that are made directly to the context instance. It is important to understand the difference, which we explain using the example of a fictitious method `modify`: To use it, type

`kk.modify(some_input)`,

but *not*

`kk2 = kk.modify(some_input)`.

Other context changes, which we present below, leave the instance intact and return a modified copy as output. Methods for modifying the incidence relation of a formal context `kk` start with the already mentioned

`kk.make_incident(g,m)`, which includes the pair `(g,m)`, and

`kk.make_nonincident(g,m)`, which removes the pair `(g,m)`. Here g and m may be object numbers or object names, but note that in the latter case they must be strings. For example, to express that the object "cow" has the attribute "livestock", you should type `kk.make_incident('cow','livestock')`. Typing `kk.make_incident(cow,livestock)` results in an error, unless you have introduced variables `cow = 'cow'` and `livestock = 'livestock'` earlier. The two commands in fact accept even sets as input: if `A` is a set of objects and `B` a set of attributes, then e.g. `kk.make_nonincident(A,B)` will remove all incidences (g, m) where $g \in$ `A` and $m \in$ `B`.

When an entire row of a context table is to be modified, it may be more convenient to use

`kk.set_intent(g,intent)`, where `g` is an object number and `intent` is a set description, either in the usual form as `{a, b, ...}`, where `a, b` are attribute numbers, or as a string of characters 'x' and '.'. Only elements in M are used.

`kk.empty()` replaces both object and attribute names by 0,1,2,... and deletes the incidence relation.

We repeat that the methods `make_incident(g,m)`, `make_nonincident(g,m)`, and `set_intent(g,'word')` have no output. They just change the context to which they are applied. No modification is caused by

`kk.isincident(g,m)`, which returns `True` iff `(g,m)` is in the incidence relation.

- Modifying a formal context "on the spot" (no return value).
  - `kk.make_incident(g,m)`
  - `kk.make_nonincident(g,m)`
  - `kk.set_intent(g,intent)`
  - `kk.trivial_labels()`
  - `kk.gname[object number] = string`
  - `kk.mname[attribute number] = string`
- Methods returning a modified context.
  - `kk2 = kk1.clarify()`
  - `kk2 = kk1.reduce()`
  - `kk2 = kk1.add_object(intent,[object name])`
- Making a new context from a given one.
  - `kk2 = kk1.copy()`
  - `kk2 = kk1.dual()`
  - `kk2 = kk1.complement()`
  - `kk2 = kk1.subcontext(gset,mset)`
  - `tc = rel.transitive_closure()`
  - `qo = kk.generators_quasiorder(extent)`
- Arrow relations and the like.
  - `da = kk.down_arrows()`
  - `ua = kk.up_arrows()`
  - `pp = kk.perspectivities()`
  - `kcl = kk.congruences()`
  - `(H,N) = kk.si_factor(m)` si = "subdirectly irreducible"

Figure 2: Modifying formal contexts.

3.6 Direct access to attributes of the `Context` class

So far no methods were implemented for editing the names of the context, the objects and the (context) attributes. The Python style guide recommends the use of *setter* and *getter* methods, but this is not yet implemented here. Instead, I do such modifications by directly addressing the variables of the `Context`

instance. So, what are they?

In our version, each formal context `kk` $= (G, M, I)$ with `gnum` $:= |G|$ objects and `mnum` $:= |M|$ attributes is encoded as follows:[4]

- The objects may have arbitrary names (i.e. strings). A list of these names is kept as the *object carrier*, named `kk.gname`. The algorithms always work with the indices of that carrier. Thus for algorithms, the object set G is always the set $\{0, 1, \ldots, g-1\}$, where $g =$ `kk.gnum` $=$ `len(kk.gname)`,
- the *attribute carrier* `kk.mname` is a list of attribute names. The working set M is always $\{0, 1, \ldots, m-1\}$ where $m =$ `kk.mnum`$=$ `len(kk.mname)`.
- The incidence relation is encoded as the list of object intents, each of which is a subset of $\{0, 1, \ldots, m-1\}$. The object intent for the object with number `h` has the name `kk.gintent[h]`.
- In addition and redundantly, the incidence is also coded as a list of attribute extents.The attribute extent for the attribute `n` is `kk.mextent[n]`. Never modify `gintent` but not `mextent`, and vice versa!
- The context name is `kk.name` of type `str`,
- the name of object `h` is `kk.gname[h]` of type `str`,
- that of attribute `n` is `kk.mname[n]` of type `str`.
- `kk.gclosure` and `kk.mclosure` contain the closure operators. This is only for convenience and may be omitted in later versions.

All these variables can be entered directly, but this is prone to error. That is why there is the method

`kk.isconsistent()`, which can be used to check whether the entered data is contradictory.

3.7 Clarifying, reducing, and enlarging a formal context

Clarifying a formal context using

`kk2 = kk1.clarify()` can considerably change the numbering, and so does reducing (which includes clarification) via

`kk2 = kk1.reduce()`.

There is a method that adds a new object, of which the object intent is given as a set `gintent` of attribute numbers, to a formal context:

`kk2 = kk1.add_object(gintent)` It allows to also specify the object's name, say `myobject`, as an optional parameter:

`kk2 = kk1.add_object(gintent, name='')`. The new object goes to the end of the object list, i.e., its object number is kk1.gnum.

The corresponding method for adding an attribute has never been implemented (although that would be easy), because it was not needed so far.

[4] Program code is listed in Section ??

3.7.1 Auxiliary

`kk2 = kk1.gclarify()` object-clarifies `kk1`

`kk2 = kk1.greduce()` object-reduces `kk1`, assuming clarified.

3.8 Create a new context from a given one

The simplest method allows to make a deep copy of a given context.

`kk2 = kk1.copy()`

A method of outputting a subcontext of a formal context is

`kk2 = kk1.subcontext(gset,mset)`. It restricts the object set to `gset` and the attributes to `mset`.

`kk2 = kk1.dual()` assigns the dual of `kk1` to the variable `kk2`.

`kk2 = kk1.complement()` assigns the complementary context of `kk1` to the variable `kk2`.

`da = kk.down_arrows()` computes the "down arrow" relation. The return value is a formal context of the same dimension as `kk`.

`ua = kk.up_arrows()` computes the "up arrow" relation.

The arrow relations can be used (among other things) for the determination of the congruence lattice of a concept lattice. The method

`pp = kk.perspectivities()` computes the (formal context of) "double down arrows". The concept lattice of the complementary context

`kcl = kk.congruences()` is isomorphic to the congruence lattice of the concept lattice of `kk`.

For every attribute `m` there is a smallest 1-generated compatible subcontext (H,N) containing `m`. This is returned when

`(H,N) = kk.si_factor(m)` is invoked. The corresponding congruence relations are exactly the ones for which the factor lattices are subdirectly irreducible.

`tc = rel.transitive_closure()`. Here `rel` must be a relation, i.e. a formal contexts with object and attribute sets of the same size (names are disregarded). The return value is the *reflexive* transitive closure of the input, thus a quasiorder.

3.8.1 Auxiliary

`kk2 = kk1.hslice(gset)` makes a subcontext, but restricts only the object set.

`kk2 = kk1.vslice(mset)` makes a subcontext, but restricts only the attribute set.

3.9 Create a new context from two given ones

`kk3 = kk1.subposition(kk2)` outputs the subposition of `kk2` under `kk1`. There is a shortcut:

`kk3 = kk1 / kk2` does the same.

`kk3 = kk1.apposition(kk2)`. Same as above, but as apposition.

`kk3 = kk1 | kk2` is the shortcut.

`kk3 = kk1.disjoint_union(kk2)` produces a formal context which contains `kk1` in the upper left quadrant and `kk2` in the lower right quadrant, with no incidences in the other two quadrants. The lists of object names are concatenated, and the lists of attribute names as well. This may lead to doublings, when the lists are not disjoint. That can be avoided using the optional parameter `disjoint`:

```
kk3 = kk1.disjoint_union(kk2,disjoint=False)
```

`kk3 = kk1.sum(kk2)` generates the context sum of `kk1` and `kk2`. As for the disjoint union, this may lead to doublings of names, when `kk1` and `kk2` are not disjoint. There is an optional parameter to avoid this:

```
kk3 = kk1.sum(kk2, disjoint=False)
```

`kk3 = kk1 + kk2` is the shortcut, but does not allow an optional parameter.

The context product is obtained by using

`kk3 = kk1.product(kk2)`, or, for short,

`kk3 = kk1 * kk2`.

For the semiproduct, use

`kk3 = kk1.semiproduct(kk2)`.

- Formal contexts arithmetics
  - `kk3 = kk1.subposition(kk2)`, or in short form
  - `kk3 = kk1 / kk2`
  - `kk3 = kk1.apposition(kk2)`, or in short form
  - `kk3 = kk1 | kk2`
  - `kk3 = kk1.disjoint_union(kk2)`
  - `kk3 = kk1.sum(kk2)`, or in short form
  - `kk3 = kk1 + kk2`
  - `kk3 = kk1.product(kk2)`, or in short form
  - `kk3 = kk1 * kk2`
  - `kk3 = kk1.semiproduct(kk2)`
  - `kk3 = kk1.substitution(kk2,g,m)` not yet implemented.
- Extents and intents
  - `B = kk.make_intent(A)`
  - `A = kk.make_extent(B)`
  - `CA = kk.close_extent(A)`
  - `CB = kk.close_intent(B)`
  - `FE = kk.first_extent()`
  - `FI = kk.first_intent()`
  - `NA = kk.next_extent(A)`
  - `NB = kk.next_intent(B)`
  - `ul = kk.uplist()`

Figure 3: Formal context operations

3.10 Extents and intents

`B = kk.make_intent(A)`, where $A \subseteq G$, returns A'.

`A = kk.make_extent(B)`, where $B \subseteq M$, returns B'.

`CA = kk.close_extent(A)`, where $A \subseteq G$, returns A''.

`CB = kk.close_intent(B)`, where $B \subseteq M$, returns B'.

`FE = kk.first_extent(aviod = set(), contain = set())` returns the extent closure of the empty set. If the two optional parameters `avoid` and `contain` are not both empty, then only extents containing `contain` and avoiding `avoid` are produced by `first_extent()` and (without repeating the optional parameters) also by `next_extent()`.

`FI = kk.first_intent(aviod = set(), contain = set())` returns the intent closure of the empty set. For the optional parameters, compare `first_extent()`.

`NA = kk.next_extent(A)` returns the lectically next extent after the set $A \subseteq G$, except when A is the last extent. Then `None` ist the return value. Warning: invoking `next_extent` without invoking `first_extent` before leads to an error, since `first_extent` initializes the closure operator.

`NB = kk.next_intent(B)` returns the lectically next intent after the set $B \subseteq M$, except when B is the last intent. Then `None` ist the return value. Warning: invoking `next_intent` without invoking `first_intent` before leads to an error, since `first_intent` initializes the closure operator.

`n = kk.number_of_concepts()` gives the number of concepts of `kk`.

`ul = kk.uplist()` produces an immutable list (a tuple) consisting of all triples `(extent, intent, parents)`, where parents is the set of `c` for which `ul[c]` is an upper neighbor. The intents are in lectic order. Te name "uplist" is misleading, the list starts at the top of the concept lattice and proceeds downwards. The syllable "up" refers to the `parents` sets, which contain the information about the upper neighbours of each entry.

`kk.successor_list()` simply prints a successor list.

`kk.number_of_maxchains()` returns the number of maximal chains in the concept lattice of `kk`.. When applied to the contraordinal scale of an order relation, the result is the number of linear extensions of that order.

When working with extents and intents we usually refer to sets of object numbers or attribute numbers, respectively. The two methods

`setvar2 = kk.gnames(setvar1)` and

`setvar2 = kk.mnames(setvar1)` show the corresponding object and attribute names.

4 Examples

The package offers more methods than discussed so far, but now it is time for some examples.

4.1 Counting monotone Boolean functions

We calculate the number of monotone Boolean functions on five variables. This is also the number of elements of FCD(5), the free complete lattices with five generators. A formal context for this lattice is obtained as the fifth power (with respect to the context product) of a 2 by 2 context with single-element incidence.

In the example below, we first produce such a 2 by 2 context and give it the name `cc`. Then we invoke `print` to see if it is correct.

```
>>> from fca\_math import *
>>> cc = Context(2,2)
>>> cc.make_incident(0,1)
>>> print(cc)
\begin{cxt} % 2 objects, 2 attributes
\cxtName{}
\att{m0}  % 0
\att{m1}  % 1
\obj{.x}{g0}  % 0
\obj{..}{g1}  % 1
\end{cxt}
```

This looks good! `cc` is a 2 by 2 context with only one incidence. Its fifth power will be a 32 by 32 context and may be too large to be printed here. So let us look at the fourth power of `cc` by invoking `print_incidence()`.

```
>>> cc.product(cc).product(cc).product(cc).print_incidence()
  .xxxxxxxxxxxxxx
  ..xxxxxxxxxxxx
  .x.xxxxxxxxxxx
  ....xxxxxxxxxx
  .xxx.xxxxxxxxx
  ..xx..xxxxxxxx
  .x.x.x.xxxxxxx
  ........xxxxxx
  .xxxxxx.xxxxxx
  ..xxxxx..xxxxx
  .x.xxxx.x.xxxx
  ....xxx.....xxx
  .xxx.xxx.xxx.xx
  ..xx..xx..xx..x
  .x.x.x.x.x.x.x
  ...............
```

The incidence relation shows a remarkable recursive structure, which is desscribed in the literature. Next we try the fifth power, using the shortcut notation:

```
>>> (cc * cc * cc * cc * cc).number_of_concepts()
7581
```

So the number of monotone Boolean functions in five variables is 7581. You may also try this for six variables, but then you will need to be patient for a few minutes before the result 7 828 354 appears. We advise against going further and calculating the concepts of the seventh power.

4.2 Do integer partition lattices have congruences?

We test whether integer partition lattices have non-trivial congruence relations. Irregularly structured lattices, as they often arise from natural data, usually have only two congruence relations and therefore only trivial homomorphic images. The lattices of integer partitions also look somewhat irregular, so one expects few congruence relations. As our object of investigation, we take the 77 partitions of the number 12. The congruence lattice is the concept lattice of the complementary context of the context `perspectivities()`.

```
>>> from fca\_math import *
>>> ip12 = Scale.integer_partitions(12)
>>> ip12.number_of_concepts()
77
>>> ip12.perspectivities().complement().number_of_concepts()
2020096
>>>
```

Wow! This small lattice has over two million congruence relations!

4.3 Size of an order relation

The order relation of a concept lattice (as a set of pairs ordered componentwise) is itself a concept lattice. In fact, with the componentwise operations, it is a complete sublattice of the direct product of the lattice with itself. It is therefore represented by a closed subrelation of the context sum of `kk` with itself. The construction is well known (see Ganter & Wille 2nd ed., p.44). As example we use the the contranominal scale on four objects.

```
>>>  cn = Scale.contranominal(4)
>>>  full = Context(4,4).complement()
>>>  order = (cn | cn) / (full | cn)
>>>  order.number_of_concepts()
81
```

Interesting! The order relation has 81 elements. Is there an easy rule behind it? To find out, we try the same for 3 and 5, and obtain 27 and 243. It looks as if the size of the order of the contranominal lattice with n atoms is 3^n. (This is indeed true and can be derived from the commutativity of the context product.)

5 More on formal contexts

5.1 Properties of formal contexts and of concept lattices

`kk.is_distributive()` returns `True` iff `kk` has a distributive concept lattice.

Calling

`kk.is_si()` returns `True` if the concept lattice of `kk` is subdirectly irreducible, `False` otherwise.

- `kk.is_distributive()`
- `is_semimodular()`
- `is_joinsemidistributive()`
- `is_meetsemidistributive()`
- `is_joindistributive()`
- `is_meetdistributive()`
- `kk.is_si()`

Figure 4: Properties of formal contexts and their concept lattices

5.2 Generators and such

`qo = kk.generators_quasiorder(A)` returns the quasiorder on G defined for an extent $A \subseteq G$ by

$$g \sqsubseteq h :\iff (A \cup \{g\})'' \subseteq (A \cup \{h\})''$$

as a formal context. The method uses the fact that

$$\not\sqsubseteq = \bigcup_{m \in A'} (G \setminus m') \times m'.$$

The minimal sets of the quasiorder correspond to the upper neigbours of A, i.e., A together with each element of such a class generates an upper neighbour extent, and these extents are equal for elements of the same class. A set of representatives is computed by the next function, `mungos(A)`.

`smungo = kk.mungos(A)`, where A is an extent of `kk`. The name "`mungo`" stands for *minimal upper neighbor generating object*. The method returns a set `smungo` of objects with the following property: For every upper neighbor U of A, `smuno` contains exactly one element g_U such that $(A \cup \{g_U\})'' = U$. Moreover, g_U has the smallest object number with this property.

Warning: not sufficiently tested.

5.3 Bonds and closed relations

An important tool for connecting formal contexts are bonds and dual bonds. The bonds from one context to another one form a closure system, which means that for appropriate subsets J one always has a *smallest bond* containing that set. The function

`B = close_bond(kk1,kk2,J)` computes this.

Bonds go from one formal context `kk1` to a second one, `kk2`. *Self-bonds*, i.e., bonds where `kk1` is equal to `kk2`, are particularly interesting. The include the *block relations*, which are the self-bonds which contain the incidence relation, and the *closed relations*, which are certain self-bonds contained in the incidence relation, which correspond to complete sublattices.

`brel = skeleton_blockrel(kk)` returns the skeleton blockrelation, which is the smallest block relation containing the neighbourhood relation. Its concept lattice is the *skeleton* of the concept lattice of `kk`.

`sp = spine(kk)` returns the closed realtion corresponding to the sublattice consisting of all formal concepts of `kk` which are comparable to all formal concepts of `kk`.

6 Closure operators

We use the `next_closure` algorithm for generating the intents and/or extents of a formal context. There are many other algorithms for that, and some of them are faster, but `next_closure` is incredibly simple and flexible, and fast enough for our purposes.

But, as the name suggests, `next_closure` is more general and is actually an algorithm for enumerating the closed sets of a given closure operator. We have set up a separate file for this, which currently contains very few procedures. However, it is conceivable that it will develop into a stand-alone package for closure operators over time.

6.1 The `Closureoperator` class

The same approach is used here as for the class `Context`: all calculations are performed over the set $\{0, 1, \ldots, size - 1\}$, but closure operators on arbitrary finite sets can be simulated. This is enabled by the class attribute `carrier`, which is a list of names of elements. The actual computations are done with the indices of that list. The command

```
clo = Closureoperator(carrier, operator)
```

introduces a new instance of this class and gives it the name `clo`. Both arguments are mandatory: `operator` must be a closure operator, and `carrier` defines the set on which it operates. This argument is either of class 'list' or of class 'int'. In the latter case it is interpreted as `size`, and the carrier is set to [0,1,...,`size`-1]. The `operator` is supposed to be a function which maps

subsets of $\{0, \ldots, \texttt{size} - 1\}$ to other subsets of that set and has the properties of a closure operator. In the case of a formal context `kk`, predefined are

```
kk.gclosure = Closureoperator(kk.gname, kk.close_extent)
```
,
and

```
kk.mclosure = Closureoperator(kk.mname, kk.close_intent)
```
.

Further attributes of the `Closureoperator` class are `self.mustcontain` and `self.search`. These are set to trivial values upon initialization and may be used when `avoid_contain` is invoked, see below.

The closure `C` of a set `X` under a closure operator `clo` is obtained by

```
C = clo.operator(X)
```
.

The method to obtain a closure operator from a given list `list_of_imps` of implications (the list must be of `class Implist`) is

```
clo = Closureoperator.from_implist(list_of_imps)
```
.

Once such an instance is defined, it may be used to browse through all closed sets using

```
First = clo.first_closure()
```
and

```
Next = clo.next_closure(Previous)
```
,
where `Previous`, `First` and `Next` are subsets of $\{0, \ldots, \texttt{size} - 1\}$. If `Previous` is the last closed set (with respect to the lectic order), then the return value is `None`.

There is a way to tweak an instance of this class so that the `first_closure`, `next_closure` combination does not generate all, but only certain closed sets. This is done by invoking

```
clo.avoid_contain(avoid, contain)
```
(no return value).

Both parameters are optional and equal to `set()` by default. The method does not modify the operator itself, but the order of the list `clo.carrier`, and the parameter `clo.search[0]`. When invoked, the subsequent calls of `first_closure` and `next_closure` will generate all closed sets which contain the set `contain` and are disjoint from `avoid`. To end this behaviour one may either re-instantiate `clo`, or invoke `clo.avoid_contain()` again, but without the optional parameters. The effects are slightly different: while a fresh version of `clo` behaves as usual, when `avoid_contain` is called again, the closed sets are all generated, but in a different order.

Input and output of the `next_closure` function are sets of indices. To see the names of the elements, use

```
names = named_set(clo.carrier,set_of_indices)
```
and

```
indices = denamed_set(clo.carrier,set_of_names)
```
for the reverse direction.

6.2 Making a formal context from a closure operator

The closed sets of any closure operator, ordered by set inclusion, form a complete lattice. We construct a formal context representing this lattice. More precisely, we determine a formal context, the extents of which are exactly the closed sets of the given operator. As object set of this formal context we take the carrier of the closure operator. As attributes we take the meet-irreducible closed sets. For that, we need a test method:

`clo.is_meet_irreducible(closedset)` returns `True` or `False`, respectively. No such test is necessary for the join-irreducibles, because each such closed set must be te closure of a singleton set (but not conversly, which implies that the resulting formal context is not necessarily object-reduced). The formal context for the closure operator `clo` then is returned when calling

`kk = clo.make_context()`.

6.3 Generators

Given a set A, one may ask if there is a proper subset of A with the same closure. We call A a *minimal generator* (or a *mingen*) if this is not the case, i.e., if every proper subset of A generates a proper subset of the closure of A. The function

`N = clo.next_mingen(A, k = 100)` returns the next mingen after A (with respect to the lectic order). The optional parameter k limits the size; only mingens of size $< K$ are considered.

`X = clo.extremal_points(inset)` returns the set of *extremal points* of the closed set which is generated by `inset`. e is an extremal point of a closed set C iff $C \setminus \{e\}$ does not generate C.

6.4 Example

It is often somewhat cumbersome to get a closure operator into the computer at all. Usually, one refers to other mathematical structures. We will demonstrate this using an example taken from a publication by Pfaltz and Jamison[5]. The authors introduce a closure operator there that could be called –very imprecisely– "overshadowing": each object is assigned its "shadow", and an object is overshadowed by a set if both the object and its shadow are covered by the set along with its shadow.

This can be made more precise using the example of (undirected, loopless) graphs (V, E): as the "shadow" of a set of vertices we define its neighbourhood, to which the set itself is also added ("closed neighbourhood"):

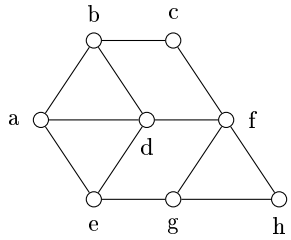
$$N[Y] := Y \cup \{v \in V \mid \exists_{y \in Y}(v, y) \in E\}.$$

With this definition,

$$\text{closure}(Y) := \{v \mid N[\{v\}] \subseteq N[Y]\}$$

[5] J.E. Pfaltz, R.E. Jamison: *Closure systems and their structure*, Information Sciences 139 (2001).

defines a closure operator.



adj	a	b	c	d	e	f	g	h
a		×		×	×			
b	×		×	×				
c		×				×		
d	×	×			×	×		
e	×			×			×	
f			×	×			×	×
g					×	×		×
h						×	×	

Figure 5: A graph and its adjacency matrix

Figure 5 shows a small graph together with its incidence matrix `adj`. Note that, for example, h is in the closure of $\{f, g\}$, but e is not, because a is a neighbour of e which ist not in $N[\{f, g\}] = \{f, g, c, d, e, h\}$.

How many sets are closed? Two simple observations help: First we add the main diagonal to the adjacency matrix:

```
>>> adjr = adj.copy()
>>> for v in range(adj.gnum):
>>>     adjr.make_incident(v,v)
```

Then the object intent `adjr.make_intent({v})` of any vertex $v \in V$ is just $N[\{v\}]$. The second observation is that for $Y \subseteq V$ always

$$N[Y] = \bigcup_{y \in Y} N[\{y\}]$$

holds. Denoting the complement $V \setminus X$ of a set $X \subseteq V$ by X^c, one obtains

$$\begin{aligned} v \in \text{closure}(Y) &\iff N[\{v\}] \subseteq N[Y] \\ &\iff N[\{v\}]^c \supseteq N[Y]^c \\ &\iff v \in \texttt{adjr.complement().close_extent(Y)}. \end{aligned}$$

Thus the closure operator is just the extent closure operator of the complementary context of `adjr`.

```
>>> print(adjr.complement().number_of_concepts())
>>> 26
```

There are 26 closed sets in the example.

7 Diagrams with given placement

Concept lattices require diagrams. However, it is not easy to draw these on a computer; it requires several steps. An essential step is to specify coordinates for the circles representing the formal concepts. It is then fairly easy to turn a given *placement* into a diagram, and we will discuss this first. Our current methods are very preliminary and will be refined or even replaced later.

A **placement** for a concept lattice is a list of coordinates specified in lectic order of the concept intents. It is stored in a `Placement` object, the only parameter of is its length, i.e., the number of formal concepts. Note that the definitions are built for a more general use, e.g., for arbitrary order diagrams or even graphs. A `Placement` object, say

`pos = Placement(7)` has three attributes, `pos.length` (which obviously contains the length), `pos.xy`, which is a list of pairs of `float16` numbers for the coordinates, and `pos.normalized`, a boolean variable which is set to `True` when the height of the diagram is set to 1.

Such a placement is usually written down in a text file with the extension `.pos`, containing for each formal concept a row with its x-coordinate and its y-coordinate, without commas or brackets, separated by one or more blanks. The order of the rows corresponds to the lectic order of the concept intents. Recall that this the same as the order of `kk.uplist()`, but also of `kk.twenty_concepts()`. There is a separate folder called **pos** where such "posfiles" are stored. To read a posfile, invoke

`pos.read('posfilename')`. To write (the normalized placement), type

`pos.write('posfilename')`, and

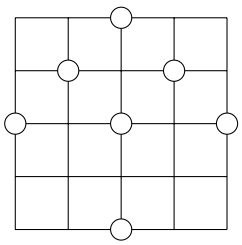
`pos.normalize()` to set the diagram heigth to 1.

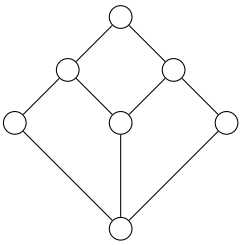
As an example we show a placement for the interordinal scale. First we invoke that scale, reduce it for simplicity, and print its formal concepts in the desired order:

```
>>> from fca_math import *
>>> kk = Scale.interordinal(3)
>>> kk = kk.reduce()
>>> kk.twenty_concepts()
0 : ( {0, 1, 2} , set() )
1 : ( {1, 2} , {2} )
2 : ( {2} , {2, 3} )
3 : ( {0, 1} , {1} )
4 : ( {1} , {1, 2} )
5 : ( {0} , {0, 1} )
6 : ( set() , {0, 1, 2, 3} ) .
```

The entries of a possible posfile is shown below at the left. The drawing in the middle shows the positioning of the seven circles. The diagram on the right

```
 0  4
 1  3
 2  2
-1  3
 0  2
-2  2
 0  0
```





is obtained as follows:

```
>>> from fca_math import *
>>> kk = Scale.interordinal(3)
>>> pos = Placement(7)
>>> pos.read(kk,'Interord3')
>>> npos = pos.normalize()
>>> latexdiagram(kk)
```

In the present version, this works in my personal setup, with `pdflatex` installed, `fca.sty` and `\tikzlibrary{fca}` available (e.g., inside the `latex` subfolder), and `okular` as the pdf-viewer.

Instead of `latexdiagram(kk)` you can also call `tkdiagram(kk)` Both methods have optional keyword arguments. In full, the functions are:

`latexdiagram(kk,placement,diagramfilename='pydiag',bluenode=set())`,

allowing to choose a file name for the diagram (the default is `pydiag`) and to give a list of node numbers to be colored (default = set()). Similarly,

`tkdiagram(kk,placement,bluenode = set(),canvas_height=500)`. The `placement` parameter, which is `None` by default, can be replaced by a suitable instance of the Placement class. When equal to `None`, the program tries to replace it by the `Context` attribute `kk.placement`.

7.1 For ConExp users

Serhiy Yevtushenko's `ConExp`, a popular computer program for Formal Concept Analysis, is still used by many although it hasn't been updated for more than twenty years. The placements of lattice diagrams produced using `ConExp` can be imported by `fca_math`, but one should carefully follow the instructions below.

We use `ConExp`'s `.csc`-Format. When editing a diagram in `ConExp`, you will find the parameter `Attribs` on the left-hand side under `Drawing options`, for which you must select the value `Show labels`. Then click on the icon with the image of a floppy disk (below the one with the hand) and select the `.csc` format to save. The file must then be moved to the `.csc` folder of the `fca_math` package, from where it can be read using

`pos = readcsc(kk,filename)`. Here `kk` denotes the formal context, while the `filename` argument is optional. If missing, `ConExp` looks for a file named `conexp.csc` inside the `csc`-folder. The function returns a normalized placement.

8 Additive diagrams

Let (G, M, I) be a formal context and assume, for simplicity, that $G \cap M = \emptyset$. The placement of an *additive* diagram of the concept lattice $\underline{\mathfrak{B}}(G, M, I)$ is obtained as follows

1. To every formal concept $(A, B) \in \underline{\mathfrak{B}}(G, M, I)$ we assign its *set representation*
$$\mathrm{rep}(A, B) := A \cup (M \setminus B).$$

2. To each element $e \in G \dot\cup M$ we assign a 2d-vector
$$\mathrm{vec}(e) = (x_e, y_e), \text{ with } y_e > 0.$$
We are free to choose these vectors. Their choice determines the diagram.

3. Then
$$\mathrm{pos}(A, B) = \sum_{e \in \mathrm{rep}(A,B)} \mathrm{vec}(e).$$

It only looks complicated at first glance. Additive diagrams have many advantages. They are often quite easy to read, but can also be processed using linear algebra. First step is to create a matrix for the set representation.

`srm = setrepmatrix(kk)` returns a $0-1$–matrix with $|\underline{\mathfrak{B}}(G, M, I)|$ rows and $|G| + |M|$ columns. So there is one row for each formal concept and one column for every object and one for every attribute. The rows are in lectic order of the concept intents, i.e., in the order of `kk.uplist()`. The order of the columns is less important, objects first, then attributes. The entry in the row correspoding to (A, B) and the column corresponding to $e \in A \dot\cup B$ is zero or one. More precisely, it is equal to one iff $e \in A$ or $(e \in M \setminus B)$, i.e., iff $e \in$ rep(A,B).

```
>>> from fca_math import *
>>> kk = Scale.interordinal(3).reduce()
>>> srmatrix = setrepmatrix(kk)
>>> print(srmatrix)
[[1. 1. 1. 1. 1. 1. 1.]
 [0. 1. 1. 1. 1. 0. 1.]
 [0. 0. 1. 1. 1. 0. 0.]
 [1. 1. 0. 1. 0. 1. 1.]
 [0. 1. 0. 1. 0. 0. 1.]
 [1. 0. 0. 0. 0. 1. 1.]
 [0. 0. 0. 0. 0. 0. 0.]]
```

From the set representation matrix one obtains co-ordinates for a placement by simple matrix multiplication with the array of 2d-vectors. The rows of the array must be in the same order as the columns of the set representation matrix. There is a function for this matrix multiplication,

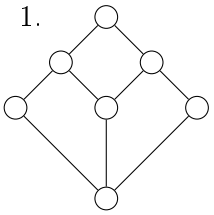
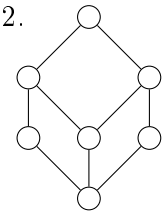
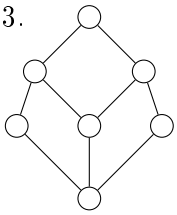
`pos = srm_evaluate(kk,srmatrix,vec)`.

```
>>> vec = np.array(
[[0, 0],
 [0, 0],
 [0, 0],
 [1, 1],
 [1, 1],
 [-1, 1],
```

```
 [-1, 1]])
>>> pos = srm_evaluate(kk,srmatrix,vec)
>>> print(pos)
[[ 0.    1.  ]
 [ 0.25  0.75]
 [ 0.5   0.5 ]
 [-0.25  0.75]
 [ 0.    0.5 ]
 [-0.5   0.5 ]
 [ 0.    0.  ]]
```

Here are the (shortened) results for three choices of `vec`, followed by the corresponding diagrams:

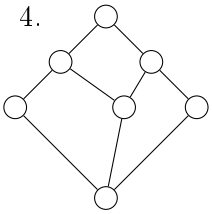
1. `vec` $[[0,0],[0,0],[0,0],[1,1],[1,1],[-1,1],[-1,1]]$
   `pos.xy` $[[0.,1.],[0.25,0.75],[0.5,0.5],[-0.25,0.75],[0.,0.5],[-0.5,0.5],[0.,0.]]$
2. `vec` $[[-2,2],[0,2],[2,2],[1,1],[1,1],[-1,1],[-1,1]]$
   `pos.xy` $[[0.,1.],[0.33,0.67],[0.33,0.33],[-0.33,0.67],[0.,0.33],[-0.33,0.33],[0.,0.]]$
3. `vec` $[[-2,2],[0,2],[2,2],[1,1],[1,1],[-1,1],[-1,1]]$
   `pos.xy` $[[0.,1.],[0.3,0.7],[0.4,0.4],[-0.3,0.7],[0.,0.4],[-0.4,0.4],[0.,0.]]$

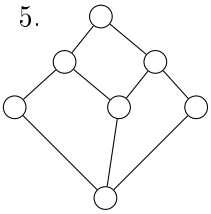
1. 
2. 
3. 

Not every diagram is additive in the sense defined above. A non-additive example is obtained when we apply a small change to the placement (1.) above, replacing the value 0. of `pos.xy[4][0]` by 0.1. This moved the node in the center of the first diagram a little to the right. The result is shown below.

How do we know that this diagram is not additive? We use linear algebra! All additive placements are obtained as a product of the set representation matrix with a suitable vector. They all therefore lie in the column space of the set representation matrix, i.e., the subvectorspace generated by the columns of this matrix. A placement therefore is additive if and only if it lies in that vector space. Even better: To any given placement there usually is a *closest* additive one, and it can be computed via orthogonaly projection to the column space of the set representation matrix. Before we clarify what we mean by "usually" here, let us look at the example where the central node is shifted.

4. `pos.xy` $[[0.,1.],[.25,.75],[.5,.5],[-.25,.75],[.1,.5],[-.5,.5],[0.,0.]]$
5. `addpos.xy` $[[-.031.],[.28.75],[.5.5],[-.22.75],[.08.5],[-.5.5],[0.0.]]$

4. 

5. 

Placement (4.) is the same as placement (1.), except that the value of `pos.xy[4][0]` was manually changed fom 0.0 to 0.1. Placement (5.) is obtained as the orthogonal projection of placement (4.) to the column space of the set representation matrix. It thus is additive. The diagrams are similar, but obviously not identical, and therefore (4.) cannot be additive.

To compute orthogonal projections, one needs an orthonormal basis, which can be computed using the Gram-Schmidt orthogonalization method.[6]

`orthomatrix = column_gram_schmidt(matrix)` returns a matrix, here named `orthomatrix`, the columns of which form an orthonormal basis of the column space of the input `matrix`. When this is done, projections can be computed using

`addpos = add_aprox(pos,orthomatrix)`, where pos is a placement, orthomatrix is as above, and the return value, `addpos`, is the orthogonal projection of `pos`. This is how (5.) above was obtained.

Caution! The output of the `add_approx` function does not automatically result in the placement of a valid lattice diagram. Similarly, the results of the `srm_evaluate` function are not always valid diagrams; this must be checked in each case. However, it is reasonable to hope that a minor change in an additive placement will again project onto an additive one. This would allow interactive dragging of nodes. However, there is still a lack of experience in this area.

9 Implications

An implication `imp` is simply a pair of sets: `imp.premise` and `imp.conclusion`. There are no further restrictions, an Implication instance, say `imp`, is introduced by stating

`imp = Implication(premise,conclusion)`, where `premise` and `conclusion` are variables of type `'set'`.

In the following example, we first introduce an implication and assign it to the variable `impvar`. Then we display this implication, printing its premise and its conclusion separately. And finally, we use the `print()` function directly for the implication `impvar`. The result is slightly different, because we have defined `print(impvar)` (via `Implication.__str__()`) to print *sorted* versions of sets.

```
>>> impvar = Implication({3,6,11,5},{7,9})
>>> print(impvar.premise,'␣==>␣',impvar.conclusion)
{3, 11, 5, 6}  ==>  {9, 7}
>>> print(impvar)
```

[6] Quote: "I prefer it over Euclidean coordinates." (from the movie *Hidden Figures*)

```
{3, 5, 6, 11} ==> {7, 9}
```

9.1 The `Implist` class

We mostly work with *lists* of implications and have an extra class for this. Here we follow the same approach that was used for contexts and closure operators: we allow arbitrary sets for the elements occuring in permises and conclusions, but internally always work with sets of the form $\{0, 1, \ldots, \texttt{size} - 1\}$. Again there is an attribute `carrier` that translates between the two representations. An instance of this class is introduced by

`impl = Implist(carrier)`, where `impl` is a name given to the object, and `carrier` is either a list of element names, or a non-negative integer `size`, which automatically is replaced by the list $[0, 1, \ldots, \texttt{size} - 1]$.

An implist is essentially just a list of implications. In the `Implist` class definition, more entries are introduced, namely those which are used for the `linclosure` algorithm. An implication `pc` is added to the implist `impl` using

`impl.addimp(pc)`.

Here is an example:

```
>>> imp1 = Implication({3,6,11,5},{7,9})
>>> imp2 = Implication({3,5},{0,1})
>>> il = Implist(23)
>>> il.addimp(imp1)
>>> il.addimp(imp2)
>>> print(il)
{3, 5, 6, 11} ==> {7, 9}
{3, 5} ==> {0, 1}
```

9.2 Closure by implications

The implications define a closure operator, which is implemented as

`Bcl = il.linclosure(B)`. Here `Bcl` is the smallest set that containes `B` and respects all implications in the implist `il`. This closure operator is also used by the `next_closure` algorithm. It can be invoked by

`Bfirst = il.first_closure()`, followed by repeated calls of

`Bnext = il.next_closure(Bprevious)`,. When the lectically largest closed set was used as input, `None` is returned.

What has been introduced here is merely a shorthand form of an already defined procedure. If an implication list `impl` is given, the corresponding closure operator can be used directly. It is obtained by

`impclosure = Closureoperator.from_implist(impl)`.

These methods suffice to compute the canonical basis of a given formal context `kk`. This is done by calling

`il = kk.canonical_basis()`.

As an example, we find the canonical basis for the Tamari lattice on four letters (the lattice has 14 elements):

```
>>> kk = Scale.tamari(4)
>>> kk.number_of_concepts()
14
>>> il = kk.canonical_basis()
>>> print(il)
{5} ==> {3,4}
{4} ==> {3}
{2} ==> {1}
{1,3} ==> {4}
{1,2,3,4} ==> {5}
{0,3,4} ==> {5}
{0,1} ==> {2}
```

Despite its theoretical elegance, the canonical basis is often not easy to read. A modification makes it more readable; this is done using the class method `human_readable()`. The following example, which continues the previous one, shows how.

```
>>> ril = il.human_readable()
>>> print(ril)
{5} ==> {4}
{4} ==> {3}
{2} ==> {1}
{1, 3} ==> {4}
{2, 4} ==> {5}
{0, 4} ==> {5}
{0, 1} ==> {2}
```

The `human_readable` method has an additional optional keyword parameter, `buddies`, which is set to `set()` by default. This can be used to determine which equivalent attributes are preferred.

You might wish to see the attribute names (which in the case of the Tamari lattice are all pairs $\{i, j\}$ of elements in $\{1, 2, \ldots, n\}$, denoted here by $i|j$.)

```
>>> print(hil.named())
{1|4} ==> {2|4}
{2|4} ==> {3|4}
{1|3} ==> {2|3}
{2|3, 3|4} ==> {2|4}
{1|3, 2|4} ==> {1|4}
{1|2, 2|4} ==> {1|4}
{1|2, 2|3} ==> {1|3}
```

9.3 Example

The readable implication base for the Tamari lattice $T(n)$ has a fairly clear form, at least in the case $n = 4$. We will therefore test whether this is also the case for larger values of n. This seems indeed to be the case, as shown

```
{1|6} ==> {2|6}
{2|6} ==> {3|6}
{3|6} ==> {4|6}
{4|6} ==> {5|6}
{1|5} ==> {2|5}
{2|5} ==> {3|5}
{3|5} ==> {4|5}
{1|4} ==> {2|4}
{2|4} ==> {3|4}
{1|3} ==> {2|3}
{4|5, 5|6} ==> {4|6}
{3|5, 4|6} ==> {3|6}
{2|5, 3|6} ==> {2|6}
{1|5, 2|6} ==> {1|6}
{3|4, 4|6} ==> {3|6}
{3|4, 4|5} ==> {3|5}
{2|4, 3|6} ==> {2|6}
{2|4, 3|5} ==> {2|5}
{1|4, 2|6} ==> {1|6}
{1|4, 2|5} ==> {1|5}
{2|3, 3|6} ==> {2|6}
{2|3, 3|5} ==> {2|5}
{2|3, 3|4} ==> {2|4}
{1|3, 2|6} ==> {1|6}
{1|3, 2|5} ==> {1|5}
{1|3, 2|4} ==> {1|4}
{1|2, 2|6} ==> {1|6}
{1|2, 2|5} ==> {1|5}
{1|2, 2|4} ==> {1|4}
{1|2, 2|3} ==> {1|3}
```

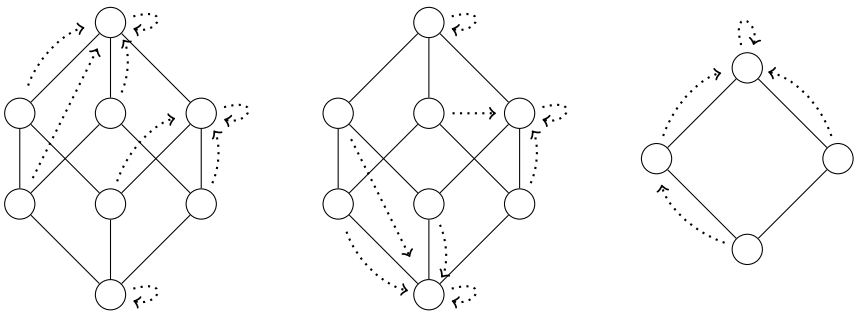
Figure 6: The readable basis for the Tamari lattice $T(6)$

in Figure 6 for $n = 6$. The reader is invited to determine some more values of these sizes of implication bases. The resulting sequence can be looked up in the Online Encyclopedia of Integer Sequences (`https://oeis.org`), where numerous interpretations are given.

10 Example: What do we need to prove?

Calculating with implications is the basis of an interactive knowledge acquisition process called **attribute exploration** or, more generally, **conceptual exploration**.[7] We will discuss this procedure in a forthcoming section. In preparation, we will look at an example that shows what this algorithm is useful for, but which does not require the interactive part of the exploration algorithm.

The example is about order-preserving self-mappings of lattices. Three examples are shown in Figure 7. φ_1 is a closure operator because it is monotone



φ_1 φ_2 φ_3

Figure 7: Three order-preserving self maps of powerset lattices.

[7] see B. Ganter, S. Obiedkov: *Conceptual Exploration*, (2016), Springer, doi:10.1007/978-3-662-49291-8

(which is synonymous with preserving order), extensive, and idempotent. φ_2 and φ_3 are not idempotent, but have other interesting properties. Here are some to consider. Let L be a lattice. An order-preserving map $\varphi : L \to L$ is called

idempotent	if	$\varphi(x) = \varphi(\varphi(x))$,	**tensive**	if	$\varphi(x) = \varphi(x \wedge \varphi(x))$,
extensive	if	$x \leq \varphi(x)$,	**increasing**	if	$\varphi(x) \leq \varphi(\varphi(x))$

holds for all $x \in L$. Idempotency is obviously self-dual, and we get three more properties by dualization: **contractive** stands for *dually extensive*, **decreasing** for *dually increasing*. There is no common word for **dually tensive**, the condition requiring that $\varphi(x) = \varphi(x \vee \varphi(x))$ for all $x \in L$. Let E_1, E_2, and E_3 be the three examples above, each consisting of the lattice and the mapping. We obtain three further examples E_1^d, E_2^d, E_3^d by dualizing. By examining all seven properties for these six examples we obtain the formal context in Figure 8. (Btw.: The context is contained in the package, you can load it with `>>> kk = cxt_predefined('opsmcl')`.) How are these seven properties

opsmcl	contractive	dually tensive	decreasing	idempotent	increasing	tensive	extensive
E1		×	×	×	×	×	×
E2			×	×	×	×	
E3					×	×	×
E1d	×	×	×	×	×	×	
E2d		×	×	×	×		
E3d	×	×	×				

Figure 8: Examples and properties of order-preserving maps.

related? Do some of them follow from others? What do we need to prove in order to gain a complete classification?
In this example, the canonical implication base provides an answer. We determine it, choosing the better readable form:

```
>>> kk = cxt_predefined('opsmcl')
>>> il = kk.canonical_basis()
>>> hil = il.human_readable()
>>> print(hil.named())
{extensive} ==> {tensive}
{tensive} ==> {increasing}
{idempotent} ==> {decreasing, increasing}
{dually tensive} ==> {decreasing}
{contractive} ==> {dually tensive}
{decreasing, increasing} ==> {idempotent}
{extensive, idempotent} ==> {dually tensive}
{contractive, idempotent} ==> {tensive}
```

It is easy to show (try it yourself!) that these implications

extensive $\Rightarrow$ tensive $\Rightarrow$ increasing,
contractive $\Rightarrow$ dually tensive $\Rightarrow$ decreasing,
idempotent $\Longleftrightarrow$ decreasing and increasing,
extensive and idempotent $\Rightarrow$ dually tensive (and the dual implication),

hold for all order-preserving self-maps of arbitrary lattices. But every implication that does *not* follow from these does not hold in the 'opsmcl' formal context and is therefore refuted by one of the six examples. The implication basis therefore provides a complete classification of the (implicational) dependencies between these seven properties. It suffices to prove the implications in the canonical basis!

We were fortunate that in this example, all implications of the canonical basis could be proven and that the three examples, together with their duals, are representative for *all* lattices. It is usually not that simple, and one must try to find counterexamples for the implications of the basis that we cannot prove. This is then the interactive process of attribute exploration.

11 Exploration

12 Many-valued contexts and conceptual scaling

13 Computing placements for diagrams

Index

Closureoperator(carrier, operator), 22
Closureoperator
    avoid_contain(avoid,contain), 22
    extremal_points(inset), 23
    first_closure(), 22
    from_implist(list), 22, 30
    is_meet_irreducible(closedset), 23
    linclosure(B), 34
    make_context(), 23
    next_closure(previous), 22
    next_mingen(setvar), 23
Context(g,m), 6
Context
    add_object(intent), 13
    apposition(kk2), 15
    canonical_basis(), 31
    clarify(), 13
    close_bond(kk1,kk2,relation), 21
    close_extent(A), 17
    close_intent(B), 17
    complement(), 14
    congruences(), 14
    copy(), 14
    disjoint_union(kk2), 15
    down_arrows(), 14
    dual(), 14
    empty(), 11
    first_extent(), 17
    first_intent(), 17
    gclarify(), 14
    generators_quasiorder(A), 20
    gflip(g,h), 7
    gnames(setvar), 18
    greduce(), 14
    hslice(gset), 15
    is_distributive(), 20
    is_si(), 20
    isconsistent(), 13
    isincident(g,m), 12
    make_extent(B), 17
    make_incident(g,m), 11
    make_intent(A), 17
    make_nonincident(g,m), 11
    mflip(m,n), 7
    mnames(setvar), 18
    mungos(A), 20
    next_extent(A), 17
    next_intent(B), 17
    number_of_concepts(), 17
    number_of_maxchains(), 17
    perspectivities(), 14
    print_incidence(), 6
    product(kk2), 15
    read('filename'), 6
    reduce(), 13
    semiproduct(kk2), 15
    si_factor(m), 14
    skeleton_blockrel(kk), 21
    spine(kk), 21
    subcontext(gset,mset), 14
    subposition(kk2), 15
    successor_list(), 17
    sum(kk2), 15
    transitive_closure(), 14
    up_arrows(), 14
    uplist(), 17
    vslice(mset), 15
    write('filename'), 6
    xdot_out(g), 11
Implication(premise,conclusion), 29
Implist(carrier), 30
Implist
    addimp(pc), 30
    first_closure(), 30
    human_readable(), 31
    linclosure(B), 30
    named(), 31
    next_closure(Previous), 30
Placement, 25
    normalize(), 25
    read(filename), 25
    write(filename), 25
Scale

additively_saturated(n), 10
biordinal(m,n), 8
contranominal(n), 8
contraordinal(rel), 10
convex_ordinal(rel), 10
dichotomic(), 10
general_interordinal(rel), 10
integer_partitions(n), 10
interordinal(n), 8
max_antichains(rel), 10
nominal(n), 8
noncrossing(n), 10
ordinal(n), 8
permutations(n), 10
pg2(d), 10
reverse_ordinal(n), 11
set_partitions(n), 10
tamari(n), 10
add_aprox(pos,orthomatrix), 29
column_gram_schmidt(matrix), 29
cxt_predefined(cxtfilename), 33
cxt_typein(), 5, 6
denamed_set(carrier,inset), 8
denamed_unknown(carrier,inset), 8
inversions(stringvar), 11
latexdiagram(kk,pos), 26
named_set(carrier,inset), 8
print(Context), 6
readcsc(kk,cscfilename), 26
set_carrier(number_or_list), 8
setinput(whatever), 11
setrepmatrix(kk), 27
srm_evaluate(kk,srmatrix,vec), 27
tkdiagram(kk,pos), 26